

Assignment 1 – Weeks 1 & 2

Database Management Systems

QueryQuest

Chandradip Karmakar

240296

Part I: Short Answer Questions

Q1. DBMS vs. Traditional File System

A traditional file system is basically a set of raw files managed by the operating system — each application owns its files and there is no centralised control. Problems pile up quickly: if two programs write to the same file at once, data gets corrupted; enforcing who can read what becomes the programmer's headache; and if the power cuts out mid-write, you may never know what got saved.

A DBMS adds a software layer that handles all of that automatically. It provides a query language (SQL), enforces access controls, guarantees that concurrent users do not step on each other's toes, and promises that a committed transaction survives even a crash (ACID properties).

Concrete scenario: Consider a hospital that records patient admissions, doctor assignments, and billing — all of which share patient data. With flat files, every department would keep its own copy, leading to inconsistent records (e.g., one file says the patient was discharged while another still marks them as admitted). A DBMS stores a single source of truth, lets doctors and billing staff query it simultaneously without conflict, and rolls back any partially completed update if the server crashes mid-transaction. A file system simply cannot offer any of these guarantees out of the box.

Q2. Weak Entity vs. Strong Entity in an ER Diagram

A strong entity has enough attributes of its own to uniquely identify every instance — it does not need help from any other entity. A weak entity, by contrast, cannot be uniquely identified on its own; it depends on a related strong entity (called the owner or identifying entity) to be meaningful.

In an ER diagram this distinction is shown visually: a strong entity uses a single rectangle, while a weak entity uses a double rectangle.

Primary key formation: A weak entity has a partial key (sometimes called a discriminator), drawn with a dashed underline. Its full primary key is formed by combining that partial key with the primary key of its owner entity.

Example: An `Order_Item` entity may only have a `line_number` as its partial key. The complete primary key becomes `(order_id, line_number)` once the owner's key is included.

Identifying relationship: The relationship connecting a weak entity to its owner is called the identifying relationship. It is drawn with a double diamond in ER notation.

Q3. Modelling Student Phone Numbers

Phone number should be modelled as a multivalued attribute.

Justification: The problem statement explicitly says a student may have multiple phone numbers. A simple attribute can hold only one value per entity instance, so it would force us to store only one number per student. A composite attribute is wrong here too — composite means the attribute is itself made up of sub-parts (e.g., an address broken into street, city, and pin code), which is not the situation. What we actually have is a single, atomic value (a phone number) that

can appear multiple times for the same student — the classic definition of a multivalued attribute, shown in ER diagrams with a double ellipse.

Q4. Candidate Keys vs. Super Keys

Super key: Any set of one or more attributes that, taken together, can uniquely identify every row in a relation. Super keys may contain extra, redundant attributes.

Candidate key: A minimal super key — one from which no attribute can be removed while still preserving uniqueness. Every candidate key is a super key, but not every super key is a candidate key.

Relation: Student(roll_no, email, name, branch)

Candidate keys:

- roll_no (each student has a unique roll number)
- email (each student has a unique email address)

Super keys that are not candidate keys:

- (roll_no, email) — uniquely identifies a row, but removing either attribute still leaves a candidate key, so this is not minimal.
- (roll_no, name) — roll_no alone already identifies the row; name is redundant.

Q5. Constraint Violated When Inserting a Non-existent Student ID

Constraint violated: Referential Integrity (Foreign Key) Constraint.

What it protects against: It ensures that every value in a foreign-key column corresponds to an existing primary-key value in the referenced table. In other words, you cannot have an Enrollment record pointing to a student that does not exist — doing so would leave a dangling reference (an orphan row) that makes the data logically inconsistent.

What should happen: The DBMS should reject the INSERT operation entirely and return an error. The Enrollment row is not written to the database, preserving the integrity of the data.

Q6. Mapping a Many-to-Many Relationship to a Relational Schema

Additional structure required: A new junction table (also called a bridge table or associative table) must be created to represent the M:N relationship.

Primary key of the junction table: It is a composite primary key made up of the primary keys of both participating entity tables. For example, if Student and Course are the two entities, the junction table Enrollment would have a composite primary key of (student_id, course_id).

Why a single column cannot represent the relationship: In a relational table, each column holds one atomic value per row. If we tried to store multiple course IDs inside a student row (or multiple student IDs inside a course row), we would violate First Normal Form and break the relational model. Moreover, storing only one foreign key in one of the tables would only capture a 1:N relationship, not M:N. The junction table solves this cleanly: each row records one pairing, so any number of students can be enrolled in any number of courses with no repetition or multi-valued columns.

Part II: Long Answer — Online Food Delivery Platform

Part A: Entity and Relationship Identification

Entities and Key Attributes

- **Customer:** customer_id (PK), name, email, phone_number(s), delivery_address
- **Restaurant:** restaurant_id (PK), name, address, cuisine_type, contact_number
- **MenuItem:** item_id (PK), name, description, price, category
- **Order:** order_id (PK), order_date, total_amount, delivery_status

Relationships and Cardinalities

- **Customer – Order (1:N):** A customer can place many orders over time, but each individual order belongs to exactly one customer.
- **Order – Restaurant (N:1):** Each order is fulfilled by exactly one restaurant, but a restaurant can receive many orders.
- **Order – MenuItem (M:N):** One order can contain multiple menu items, and the same menu item can appear in many different orders — requiring a junction table.

Part B: Attribute Classification

Multivalued attribute — phone_number on Customer:

A customer might register both a mobile number and a landline (or multiple mobile numbers). Because a single customer entity can be associated with more than one phone number value, this is a multivalued attribute. In an ER diagram it is shown with a double ellipse.

Composite attribute — delivery_address on Customer:

An address is naturally composed of sub-parts: street, city, state, and pin code. Each sub-part has independent meaning and may be queried individually (e.g., 'find all customers in Kanpur'). Modelling it as a composite attribute captures this internal structure, whereas a simple string would hide it.

Part C: Relational Schema for the Junction Table

The M:N relationship between Order and MenuItem requires the following junction table:

OrderItem(order_id, item_id, quantity, unit_price)

Primary Key: (order_id, item_id) — the combination of both foreign keys uniquely identifies each line in an order.

Foreign Keys:

- order_id references Order(order_id)
- item_id references MenuItem(item_id)

Additional attributes:

- **quantity:** how many units of this item were ordered (e.g., 3 burgers).
- **unit_price:** the price of the item at the time the order was placed, stored here because menu prices can change later.

Part III: ER Diagram Design — Hospital Database

Question 1: Hospital ER Diagram

The ER diagram below models the hospital database with three entities: Patient, Doctor, and Ward.

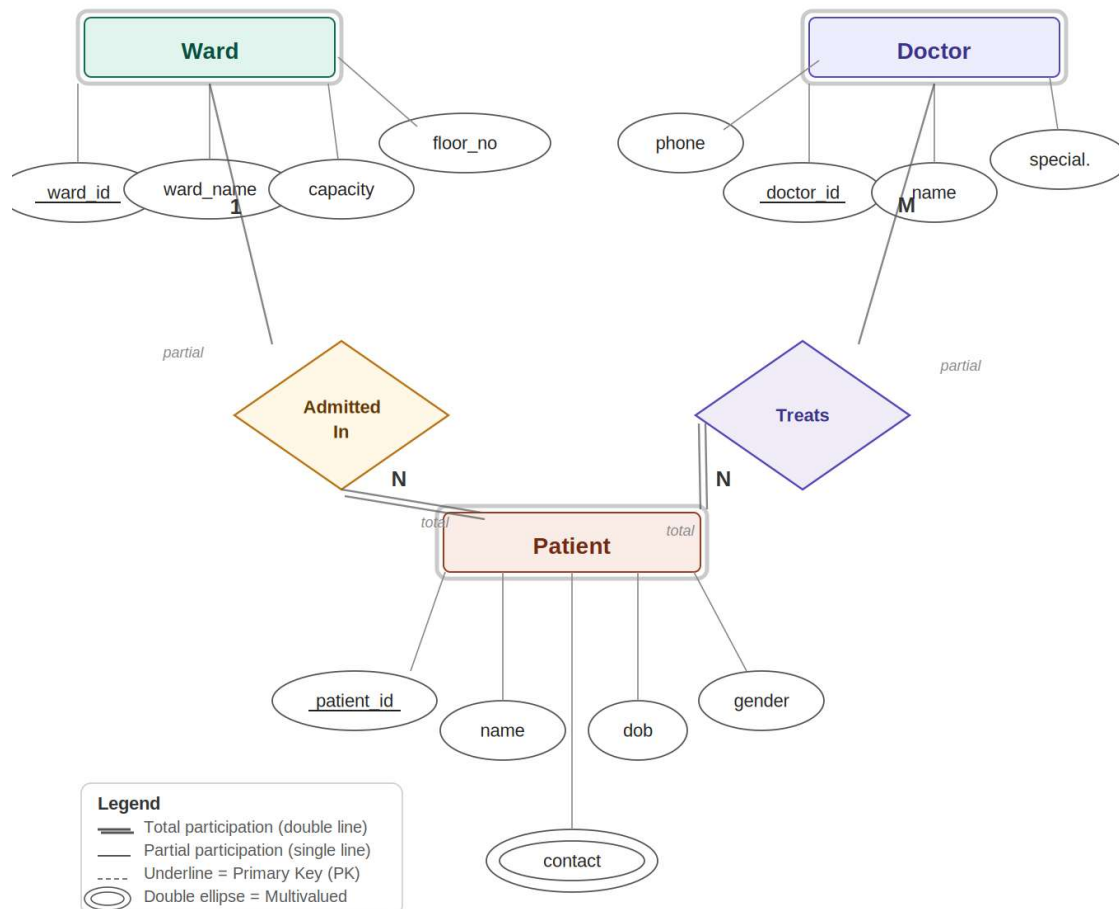


Figure 1: Hospital ER Diagram — Patient, Doctor, Ward

Question 2: Design Justification

Doctor–Patient: Why M:N and the Associative Entity

The relationship is M:N because real hospital practice works exactly that way: a single patient admitted with multiple conditions might be seen by a cardiologist, a neurologist, and a general physician simultaneously, while each of those doctors is concurrently treating several other patients. Neither side is limited to one partner on the other side, which is the definition of M:N. To represent this in a relational schema, the relationship is converted into an associative entity called Treatment (or Consultation). Its composite primary key consists of (doctor_id, patient_id). Additional attributes such as treatment_date and diagnosis_notes can be attached to this associative entity because they describe the specific interaction between a doctor and a patient, not either entity alone.

Participation Constraints

Ward – Patient (Admitted In):

- **Patient side — total participation:** In this hospital system, a patient record is created only upon admission, so every patient must be assigned to a ward. There is no scenario where a patient exists in the database without a ward, hence double lines in the ER diagram.
- **Ward side — partial participation:** A ward can exist in the system (it has a bed count and a floor number) even when it is currently empty — perhaps it was just set up or is being cleaned. So not every ward must have a patient at all times, hence single lines.

Doctor – Patient (Treats):

- **Doctor side — partial participation:** A doctor might be on leave or newly registered in the system without having been assigned to any patient yet.
- **Patient side — total participation:** Every admitted patient must be under the care of at least one doctor; an admission without a treating physician would be clinically unsafe and is not allowed by the hospital's rules.

Attribute Design Decision

The address of a Patient was modelled as a composite attribute broken into street, city, state, and pin_code. The reason is practical: the hospital may need to filter or group patients by city (for regional reporting), look up pin codes for delivery of discharge papers, or validate that the state is correct. Storing the whole address as a single string would make all of these operations require string parsing, which is error-prone and slow. Splitting it into its natural components allows direct, efficient querying of any sub-part while still reading the full address when needed.